

Autocomplete:

Dictionary $\Rightarrow$ all existing words given.

given a prefix

$\hookrightarrow$ Return a set of possibilities matching the prefix!

Naive: Iterate over the set of words that exist. Match the prefix $O(N)$ with every word

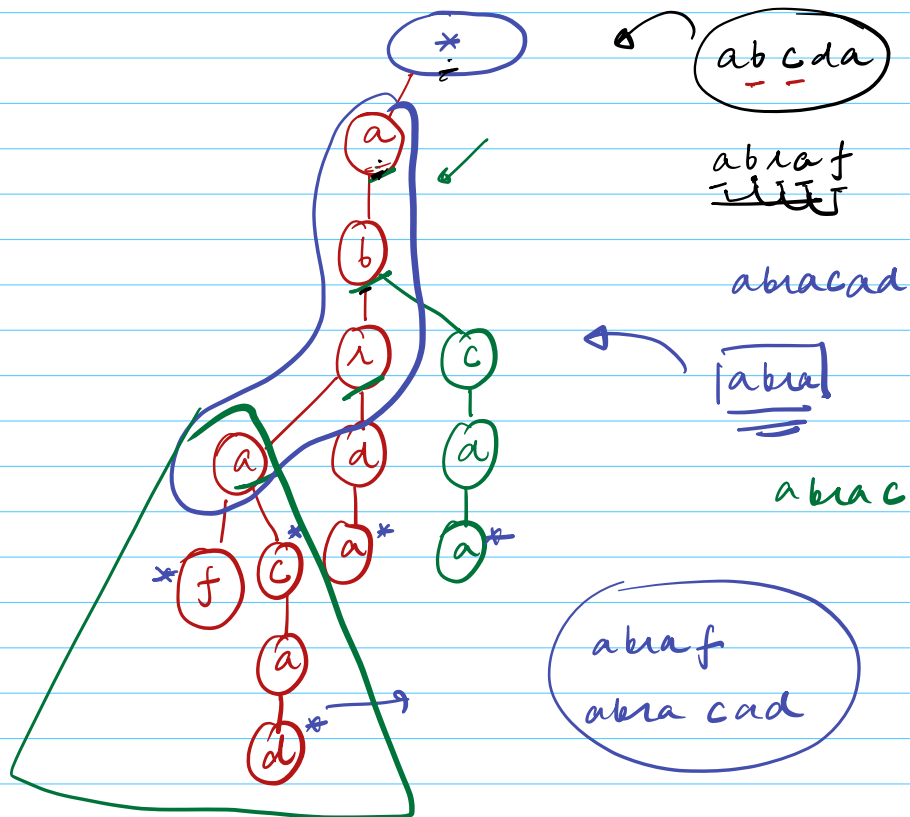
$O(\text{no. of words} \times \text{length})$

abrac

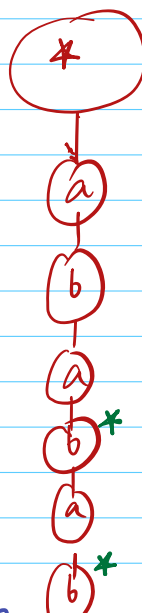
↓ ↓ ↓ ↓
a b r d a
a b r f b
a b r c k
a b r b a
a b r b c a

Tree $\Rightarrow$ n children Tree (N-ary tree)

$\hookrightarrow$ prefix tree
 $\equiv$



a b a b
 a b a b a b



int size[26]

alphabet

apple
cat
box

call
bat

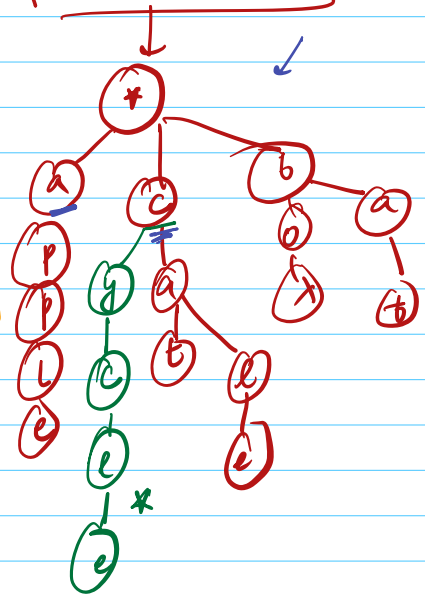
class Node {

char data;

HashMap < char, Node >

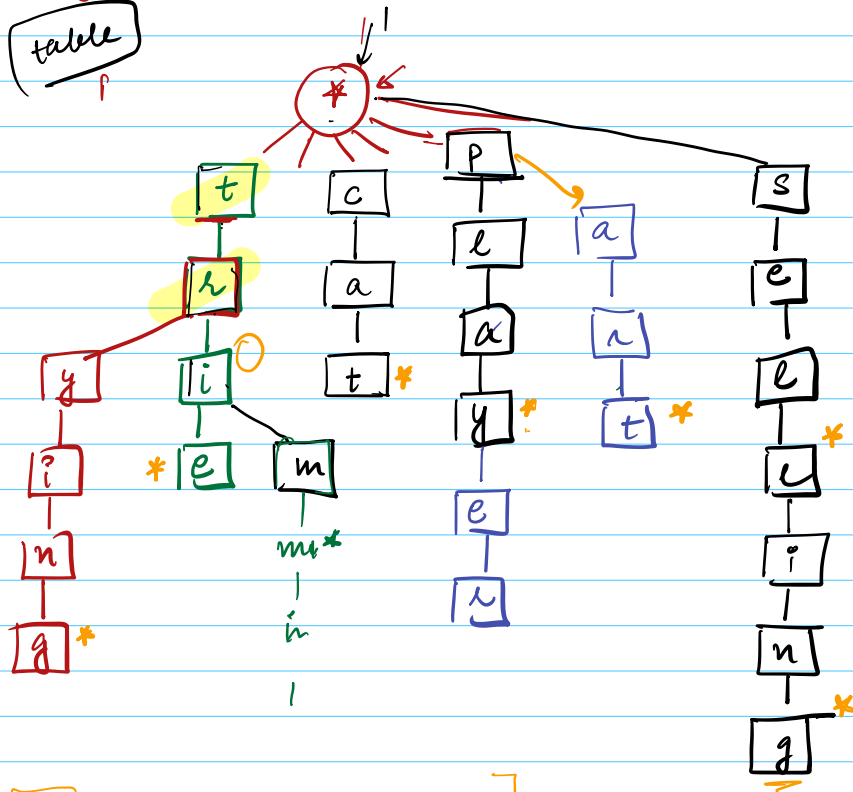
boolean end

denotes
if a
word
ends
at the
current
node



table

cat ⇒
play
part
tree
trying
trim
player



search (apple) → False
search (map)
search (tri)

TC: $O(N)$ SC: $O(N)$

Node tmpRoot = new Node('*')

void insert (Node root, string word)

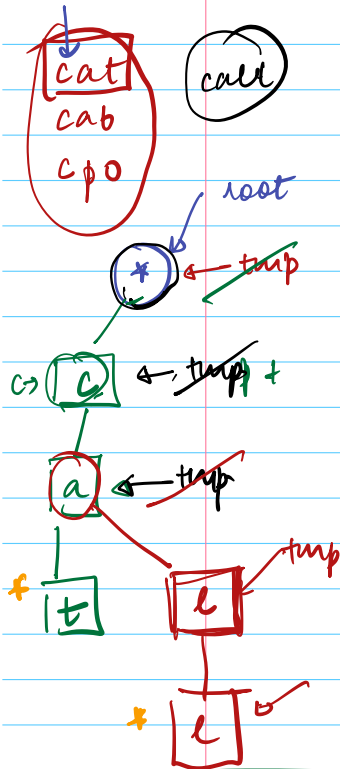
Node tmp = root;

for (char ch : word)

if (!tmp.children.containsKey(ch))
tmp.children.put (ch,
new Node(ch))

tmp = tmp.children.get (ch)

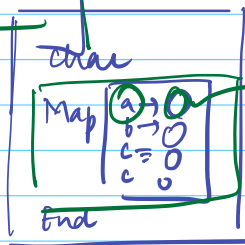
tmp.isEnd = True



```
class Node {
    char data;
    HashMap < char, Node > children;
    boolean end
}
```

HashMap < int, char > mp

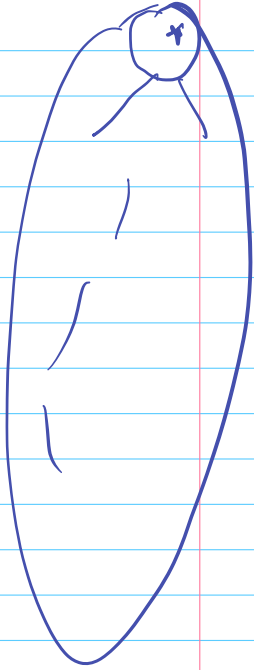
mp.get (5)



tmp.children.get ('a')

$TC: O(\text{word length})$

$SC: O(1)$



bool search(Node root, string word)

Node tmp = root

for (char ch: word)

if (!tmp.children.find(ch))
return false

tmp = tmp.children.get(ch)

return tmp.isEnd

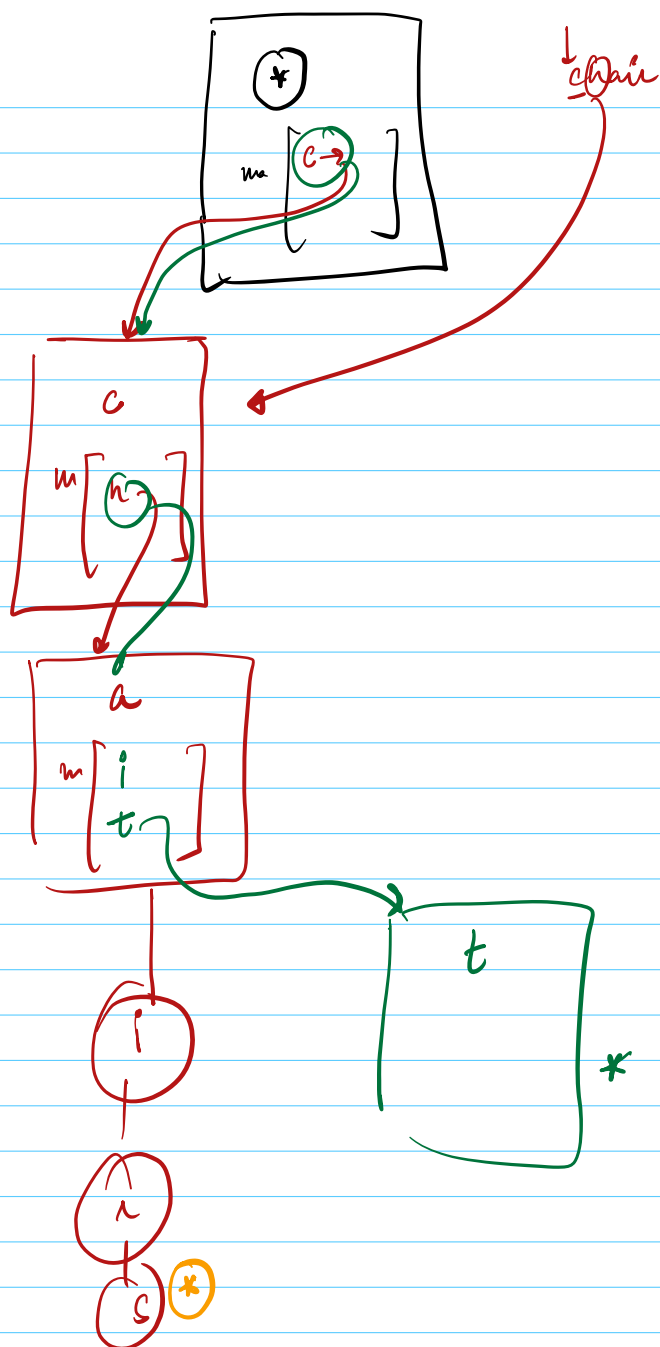
Fastest way to delete a word in Trie



Set

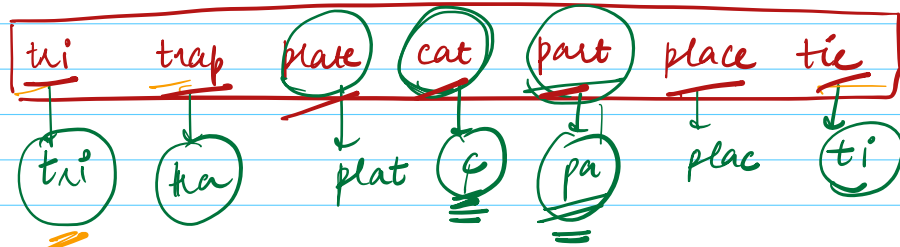
isEnd = false

last char →



3 mins

Find shortest unique prefix to represent each word.



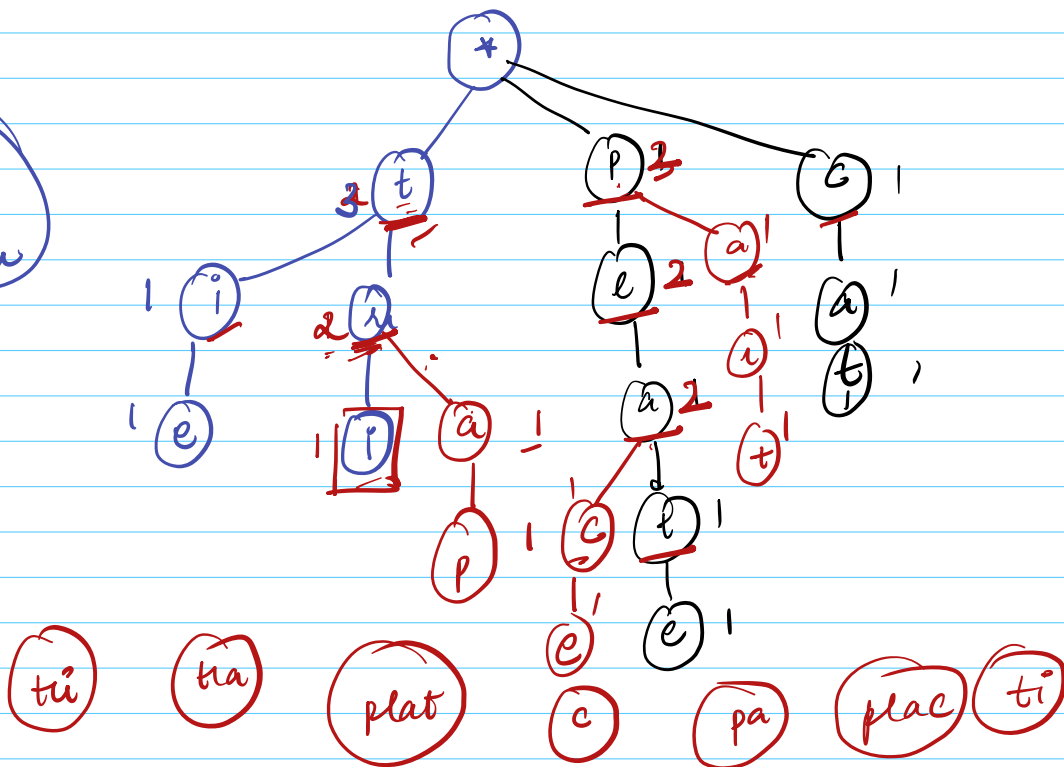
tr

prefix can only MEAN one word

Hint 1: When does a prefix become unique?

When there is only one $isEnd = True$ in the subtree of the Node

count
no. of words in the subtree



AUTO COMPLETE

↳ N words given ↳ Every word has a weight

Every word has a

Weight

5 mins

10 words prefixes

↳ For every prefix Return ATMOST 5 words in decreasing order of weight

abcd

accd

abaa

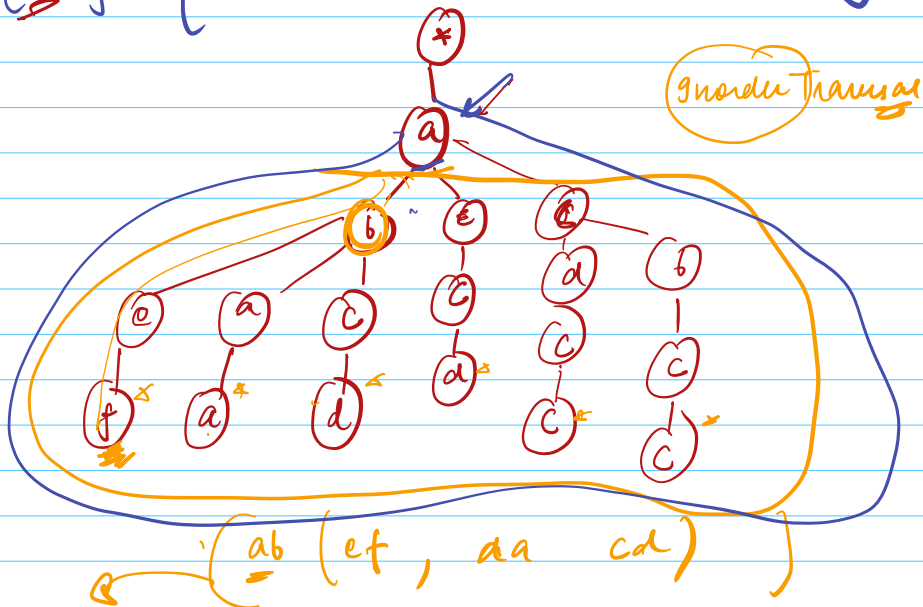
abef

1acdcc

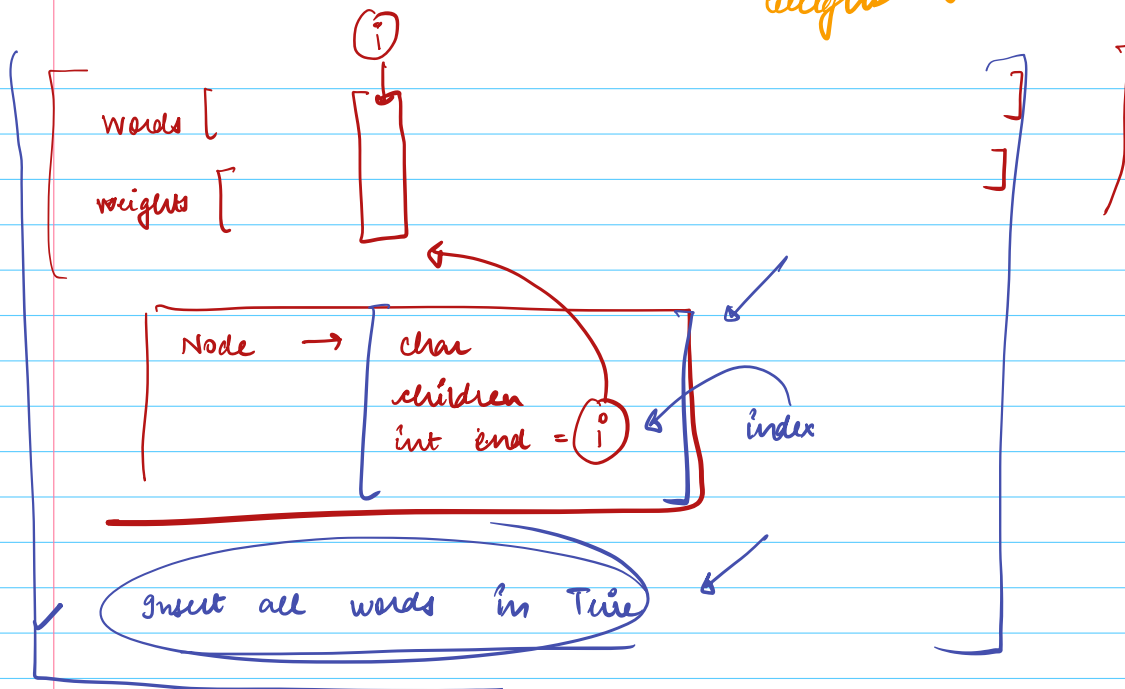
acbcc

$$\begin{pmatrix} \hat{a} \\ \hat{b} \end{pmatrix}$$

abef, abaa, abcd

$$\begin{bmatrix} a \\ \cancel{b} \end{bmatrix} \rightarrow \begin{bmatrix} \end{bmatrix}$$


sort them by decreasing order of weight



Insert from prefix

```

tmp = root
for (char ch : prefix)
{
    tmp = tmp.children.get(ch)
}

```

Inserts a word & ends at null to the list

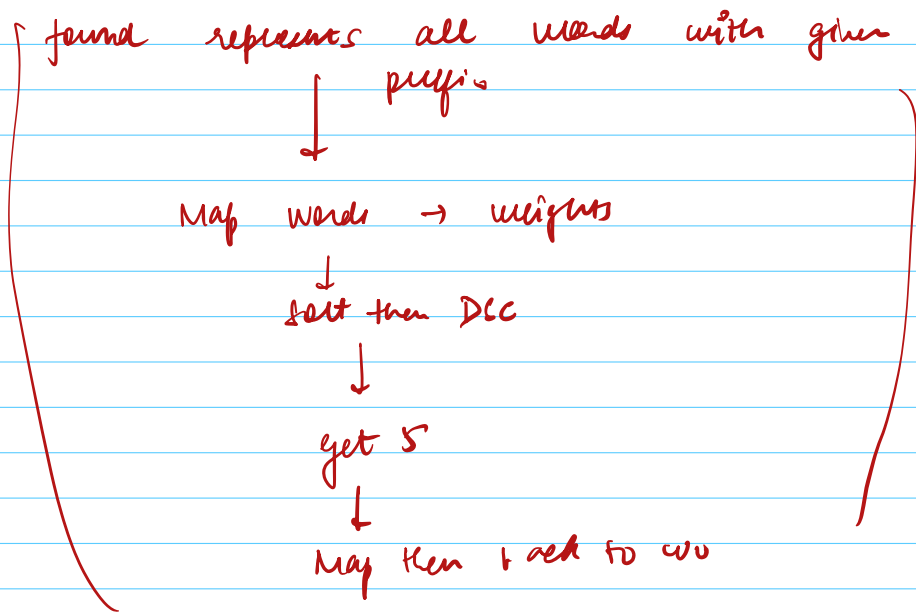
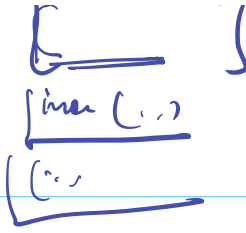
Inserts over all children of

```

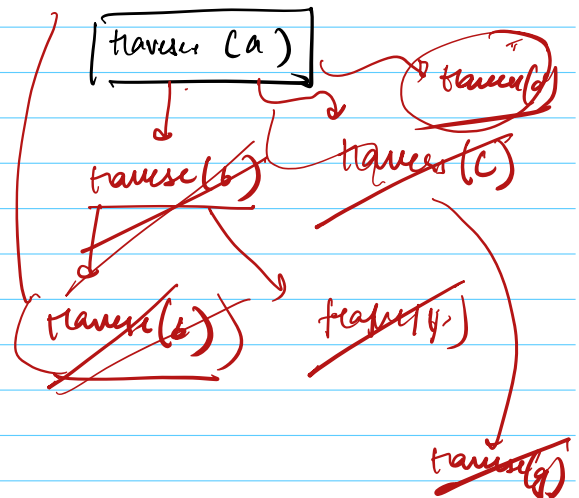
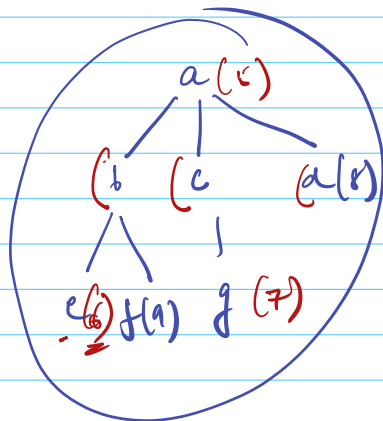
found = []
void traverse (Node cur)
{
    if (cur.end != -1)
        found.insert(cur.end)
    for (char, Node & : children)
        traverse(Node)
}

```

current word



[5, 6, 9, 7, 8]



Solu - 2

→ sort before (desc)

→ while insertion, at every node

maintain list [5]

